Systemsoftware WiSe 2026/27 – Übung 1

Jo Weydert – Matrikelnummer: 1687610

Aufgabe 1

Programm: Ein Programm ist einfach Code, der als Datei gespeichert ist. Es macht nichts, solange es nicht gestartet wird. Wenn man es startet, wird daraus ein Prozess.

Prozess: Ein Prozess ist ein laufendes Programm. Er hat einen eigenen Adressraum, auf den andere Prozesse nicht zugreifen können. In jedem Prozess läuft mindestens ein Thread.

Thread: Ein Thread ist ein Kontrollfluss innerhalb eines Prozesses, also das, was die Befehle wirklich ausführt. Jeder Thread hat seinen eigenen Stack und eigene Register, aber alle Threads in einem Prozess teilen sich den Speicher. Ein normales sequentielles Programm hat nur einen Thread.

Aufgabe 2

a)

Mehr Kerne helfen nur bei dem Teil des Programms, der parallel laufen kann. Der sequentielle Teil läuft trotzdem nur auf einem Kern und wird nicht schneller. Außerdem kostet es Zeit, die Threads zu verwalten und zu synchronisieren, und alle Kerne müssen auf denselben Speicher und dieselbe Festplatte warten.

Den Takt kann man auch nicht beliebig erhöhen, weil der Prozessor dann zu heiß wird (Abwärme). Und wenn die CPU auf den Speicher oder auf I/O wartet, bringt ein höherer Takt auch nichts.

b)

Amdahlsches Gesetz mit sequentiellem Anteil s, parallelem Anteil $p = 1 - s$ und n Kernen:

$$S(n) = \frac{1}{s + \frac{p}{n}}$$

Beispiel für $p = 0{,}75$ und $n = 4$:

$$S(4) = \frac{1}{0{,}25 + \frac{0{,}75}{4}} = \frac{1}{0{,}25 + 0{,}1875} = \frac{1}{0{,}4375} \approx 2{,}29$$

p	2 Kerne	4 Kerne	8 Kerne	16 Kerne
25 %	1,14	1,23	1,28	1,31
50 %	1,33	1,60	1,78	1,88
75 %	1,60	2,29	2,91	3,37

Man sieht, dass der Speedup mit mehr Kernen immer weniger wächst. Er kann nie größer als $1/s$ werden (z. B. maximal 4 bei $p = 75\,\%$).

Aufgabe 3

1) Ich habe `Counter.java` auf dem Quellcode vom Spring Framework (9294 `.java`-Dateien) getestet. Ergebnis: 1.528.936 Zeilen in ca. 518 ms.

2) In `ParallelCounter.java` werden zuerst alle passenden Dateien in einer Liste gesammelt. Dann werden sie über einen Thread-Pool auf n Threads aufgeteilt. Jeder Thread zählt die Zeilen seiner Dateien und gibt eine Teilsumme zurück. Am Ende werden alle Teilsummen addiert. Weil jeder Thread seine eigene Variable hat, braucht man keine Locks.

3) Beobachtungen:

- Die parallele Version war nicht schneller als das Original. Das Durchsuchen der Ordner läuft immer noch sequentiell (ungefähr ein Drittel der Zeit), und das Programm wartet hauptsächlich auf die Festplatte statt zu rechnen.
- Mit mehr Threads als Kernen wurde es deutlich langsamer, weil das Betriebssystem ständig zwischen den Threads wechseln muss.
- Am sinnvollsten ist ungefähr so viele Threads wie Kerne.